1.1.3. Minimum Spanning Tree using Prim'… 11:40



Write a C program to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Prim's Algorithm.

You are given a partially implemented driver code. Your task is to complete the missing logic in the **minKey(), prim()** functions. The program represents the graph using an adjacency matrix.

Functions to be Completed

1. minKey(): This function selects the vertex that has the minimum key value from the set of vertices not yet included in the MST.

```
int minKey(int key[], bool mstSet[], int vertices)
```

- **key[]** stores the minimum weight edge connecting a vertex to the MST.
- **mstSet[]** keeps track of vertices already included in the MST.
- The function returns the index of the vertex with the smallest key value that is not yet in the MST.

2. prim(): This function implements Prim's Algorithm to construct the Minimum Spanning Tree.

```
void prim(int graph[V][V], int vertices)
```

- This function finds the MST of a graph by starting from one vertex and repeatedly adding the nearest (lowest weight) vertex until all vertices are connected.

Input Format:

- The program first prompts the user to enter an integer vertices represents the number of vertices in the following format:

```
No of vertices:
```

- Next line, the program prompts:

```
Adjacency matrix elements (row wise):
```

- The next vertices lines each contain vertices space-separated integers representing the adjacency matrix.
- The value at row i and column j denotes the weight of the edge between vertex i and vertex j.
- A value of 0 indicates that there is no edge between the corresponding vertices.

Output Format:

- After constructing the Minimum Spanning Tree, the program prints the header:

```
Edge  Weight
```

- Then, for each edge selected in the MST, print:

```
<u> - <v>  <weight>
```

Where:

Explorer

C minCostF…

Submit

Debugger

```c
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#define V 100

int minKey(int key[], bool mstSet[], int vertices) {
    // Write your code here...
    int min = INT_MAX, min_index;

    for (int v = 0; v < vertices; v++) {
        if(mstSet[v] == false && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;


}
void printTree(int parent[], int graph[V][V], int vertices) {
    printf("Edge \tWeight\n");
    for (int i = 1; i < vertices; i++)
        printf("%d - %d \t%d \n", parent[i], i, graph[i][parent[i]]);
}

void prim(int graph[V][V], int vertices) {
    int parent[V];
    int key[V];
    bool mstSet[V];

    for(int i = 0; i < vertices; i++){
        key[i] = INT_MAX;
        mstSet[i] = false;
    }
    key[0] = 0;
    parent[0] = -1;

    for(int count = 0; count < vertices - 1; count++){
        int u = minKey(key, mstSet, vertices);
        mstSet[u] = true;

        for(int v = 0; v < vertices; v++){
            if(graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]){
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printTree(parent, graph, vertices);

```

Terminal Test cases

‹ Prev Reset Submit Next ›